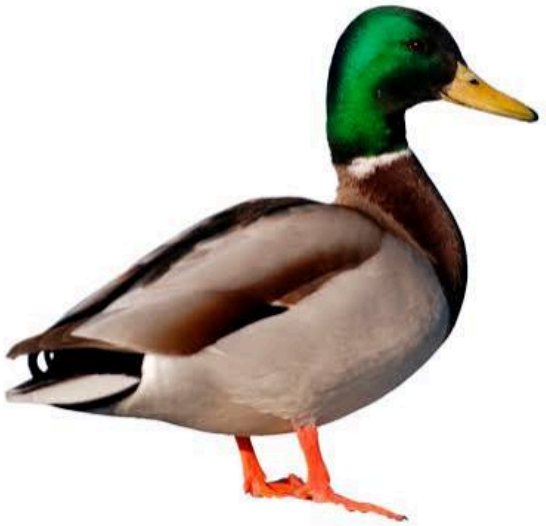
Ciência de Dados Aplicada ao Direito I

Aula 17

Bibliotecas e classes



Prof. Bruno da Cunha de Oliveira - 2026/2

Atendimento desta semana: segunda, 19/10

O horário de atendimento desta semana passa de quinta para segunda, 19/10, das 12h30 às 14h.

A revisão do que o Desafio V pede fica para esse atendimento. A sala e a sugestão de exercícios vão no Blackboard.

Para terça, 20/10: conta no GitHub e Git instalado

Na terça vamos usar o Git e o GitHub. Cheguem com duas coisas feitas:

#	O que fazer
1	Instalar o Git no seu computador
2	Criar uma conta em `github.com`

O guia está no Blackboard, em versão para Windows e para Mac: são os passos 1 e 2. Quem travar faz comigo no começo da aula.

Desafio V, no PrairieLearn: para entregar até 21/10

Vale 0,1 ponto extra na prova final para quem concluir até o início da aula de quarta, 21/10, com todas as questões aceitas pelo corretor. Dá para tirar dúvida no horário de atendimento.

Insper

Um programa de verdade

Vale 0,1 ponto extra na prova final para quem enviar pelo Blackboard, até o fim deste bloco, a explicação de pelo menos três linhas do programa, com as próprias palavras.

Um programa com o juscraper

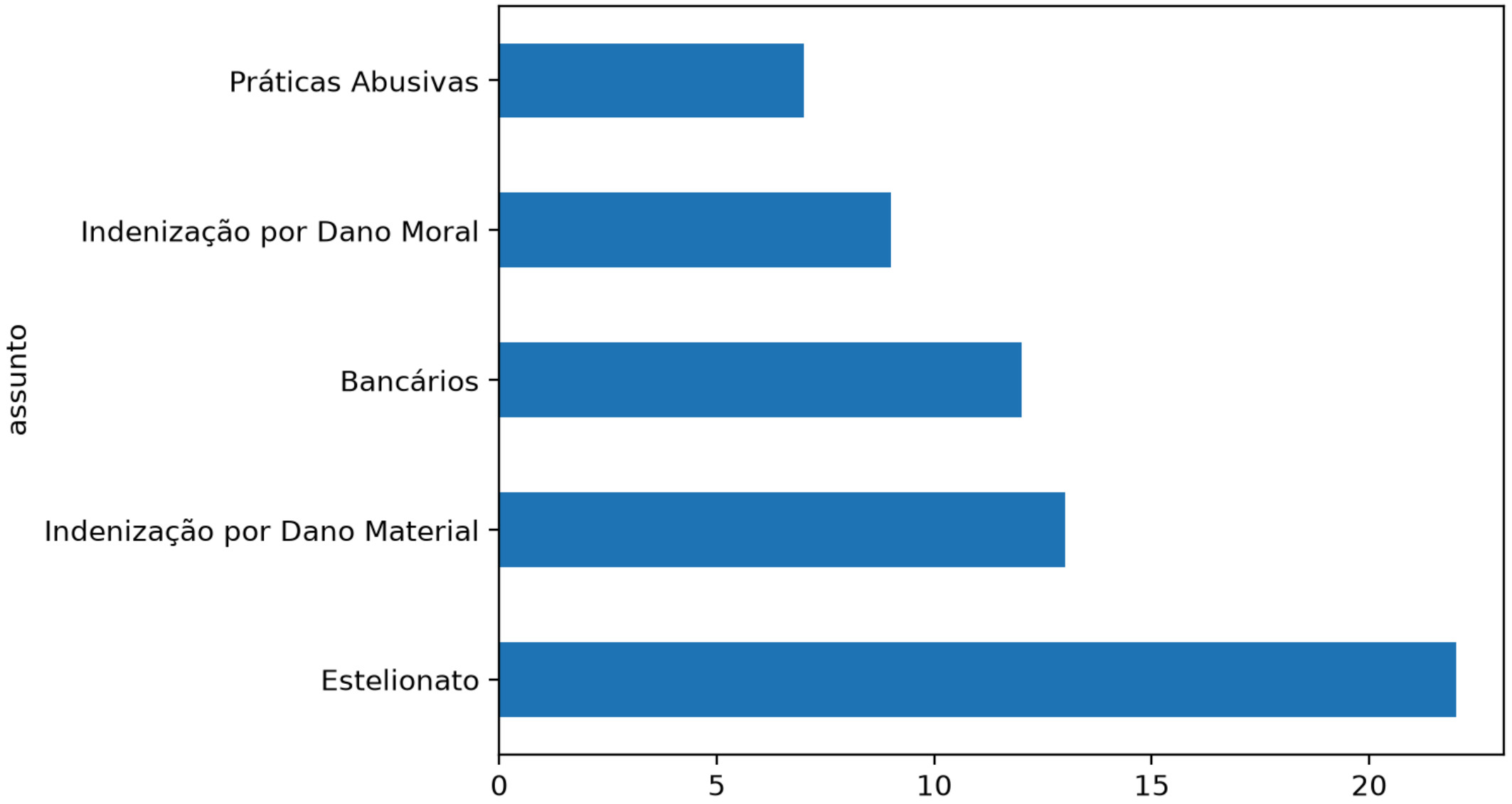
```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

Expliquem pelo Blackboard, com as próprias palavras, o que vocês acham que cada linha faz.

O gráfico que esse programa produz



Rodar a primeira linha

```
import juscraper as jus
```

Rodem esta linha num notebook da pasta `cdad1`. O que aparece?

O Python não conhece o juscraper

RESPOSTA

```
ModuleNotFoundError: No module named 'juscraper'
```

O Python que vocês instalaram não sabe o que é `juscraper`. A primeira linha do programa já não roda.


SONY
Discman
GAME BOY COLOR
TYCO
TYCO
27MHz


Oct 28, Oct 28
10:09

Python não vem com pilha

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

O que vem e o que não vem com o Python

VEM COM O PYTHON

Texto, número, lista e dicionário. `if`, `for` e funções.

É tudo o que vocês usaram até agora.

NÃO VEM

Tabelas. Gráficos. Leitura de PDF. Coleta de decisões em tribunais. Modelos de linguagem.

É quase tudo o que um trabalho com dados precisa.

Bibliotecas

O que não vem com o Python se instala.

São as bibliotecas: código que outra pessoa escreveu e publicou. O juscraper é uma delas.

Instalar o juscraper

No VS Code, abram o terminal na pasta `cdad1` e rodem:

```
uv add juscraper
```

O uv baixa a biblioteca e tudo de que ela precisa. Quem travar me chama.

Rodar de novo, com o juscraper instalado

```
import juscraper as jus
tjsp = jus.scraper("tjsp")
```

Rodem estas duas linhas. O que aparece?

A terceira linha do programa pede dez páginas ao site do TJSP, e não roda em sala.

Nada aparece, e está certo

RESPOSTA

O `import` achou a biblioteca, e `jus` é o apelido dela.

`jus.scraper("tjsp")` chama uma função da biblioteca, que devolve o raspador do TJSP, guardado em `tjsp`. Nenhuma das duas linhas mostra nada.

Entendendo o código de bibliotecas

Insper

De bibliotecas a objetos

1 Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

De bibliotecas a objetos

1. Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

2. Para entender como uma biblioteca funciona, precisamos entender objetos. É em objetos que quase toda biblioteca entrega o que faz: o coletor é um objeto, a tabela é um objeto.

De bibliotecas a objetos

1. Para fazer coisas mais interessantes, usamos bibliotecas. Ninguém reescreve, a cada trabalho, um coletor de decisões de tribunal ou uma tabela de dados.

2. Para entender como uma biblioteca funciona, precisamos entender objetos. É em objetos que quase toda biblioteca entrega o que faz: o coletor é um objeto, a tabela é um objeto.

3. Saber que `dados.shape` guarda um dado e que `contagem.head(5)` executa uma ação é o que permite ler a documentação de uma biblioteca, a mensagem de erro e o código que a IA escreve.

O ponto depois de um texto

```
tribunal = "TJSP"
print(tribunal.lower())
print(tribunal)
```

O que aparece na tela?

Um método: o texto sabe virar minúsculas

RESPOSTA

```
tjsp
TJSP
```

O ponto pede uma ação ao valor que vem antes dele: `tribunal.lower()` devolve o texto em minúsculas. Como o `.replace()`, ele devolve um texto novo, e `tribunal` continua como estava. Essa ação se chama método.

Métodos de texto

Chamada	Devolve	O que faz
`"TJSP".lower()`	`tjsp`	passa para minúsculas
`"tjsp".upper()`	`TJSP`	passa para maiúsculas
`"  TJSP  ".strip()`	`TJSP`	tira os espaços das pontas
`"TJSP".replace("SP", "RJ")`	`TJRJ`	troca um trecho por outro

Vocês já usavam métodos: `.replace()` é do texto, e `.append()`, da lista.

O ponto depois de uma lista

```
pecas = ["petição inicial", "contestação"]
pecas.append("sentença")
print(pecas)
print(pecas.lower())
```

O que aparece na tela?

Cada tipo tem os seus métodos

RESPOSTA

```
['petição inicial', 'contestação', 'sentença']
AttributeError: 'list' object has no attribute 'lower'
```

`.append()` é método de lista, e `.lower()`, de texto. Uma lista não sabe virar minúsculas, e o Python diz isso no erro: a lista não tem `lower`.

Função ou método

Elemento	Como se usa	Exemplos
função	vive solta, e o valor entra entre os parênteses	`len(pecas)`, `print(pecas)`
método	mora dentro de um tipo, e age sobre o valor antes do ponto	`pecas.append("sentença")`, `tribunal.lower()`

Um método é uma função que já sabe sobre qual valor vai agir.

Criar um tipo

O molde e os cachorros

A CLASSE É O MOLDE

```
CLASSE Cachorro:
    nome
    idade
    latir()
```

CADA CACHORRO É UM OBJETO

```
rex:  nome "Rex",  idade 3
toto: nome "Totó", idade 8
```

O molde é escrito uma vez. Cada cachorro sai dele com os seus valores. Texto e lista são tipos que alguém escreveu assim, e dá para escrever os nossos.

class e `__init__`

```python
class Cachorro:
    def __init__(self, nome, idade):
```

`class` cria o tipo e dá o nome a ele: é o molde.

`__init__` é uma função como as que vocês já escrevem, com um nome reservado. O Python a chama sozinho, toda vez que um cachorro é criado: é ela que preenche a ficha dele.

self: este cachorro

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
```

`self` é o cachorro que está sendo criado.

`self.nome = nome` guarda neste cachorro o nome que chegou pela chamada.
`self.idade = idade` faz o mesmo com a idade.

Criar dois cachorros

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

rex = Cachorro("Rex", 3)
toto = Cachorro("Totó", 8)
print(rex.idade, toto.idade)
```

O que aparece na tela?

Atributo: o que cada objeto guarda

RESPOSTA

```
3 8
```

`Cachorro("Rex", 3)` chama o `__init__`: `"Rex"` vai para `nome`, e 3, para `idade`. Quem chama não escreve o `self`. `rex.idade` lê o que ficou guardado no Rex: é um atributo, e cada cachorro tem os seus.

Um cachorro que late

```
class Cachorro:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def latir(self):
        return self.nome + ": au au"
toto = Cachorro("Totó", 8)
print(toto.latir())
```

Acrescentei uma função dentro da classe. O que aparece na tela?

Método: uma função dentro da classe

RESPOSTA

```
Totó: au au
```

Uma função escrita dentro da classe é um método. Em `toto.latir()`, `self` é o `toto`, e `self.nome` vale `"Totó"`. É o mesmo ponto de `tribunal.lower()`: pede uma ação ao objeto.

Atributo e método

```
toto = Cachorro("Totó", 8)
```

Elemento	O que é	Neste caso
atributo	um dado que o objeto guarda; sem parênteses	`toto.idade`
método	uma ação que o objeto executa; com parênteses	`toto.latir()`

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

COM VARIÁVEIS SOLTAS

`numero`, `tribunal` e `resultado` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

Programação orientada a objetos

Os dados e o que se faz com eles andam juntos, num objeto.

COM VARIÁVEIS SOLTAS

`numero`, `tribunal` e `resultado` de duzentos processos são seiscentas variáveis para não misturar. No objeto, cada processo carrega as suas.

COM FUNÇÕES SOLTAS

Quem chama precisa passar tudo, e saber qual função serve para qual dado. O método já sabe onde está: `toto.latir()`.

A ficha da parte

```
class Parte:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade
    def tem_prioridade(self):
        return self.idade >= 60
autora = Parte("Ana Lima", 66)
print(autora.nome)
print(autora.tem_prioridade())
```

O que aparece na tela?

Criar uma parte, passo a passo

Animação no navegador: `animacao-objetos.html`, cenário `criar`.

O método usa os atributos do objeto

RESPOSTA

```
Ana Lima
True
```

Em `autora.tem_prioridade()`, `self` é a `autora`, e `self.idade` vale 66, que passa de 60.

Na autuação, alguém preenche a ficha de cada parte. A classe é a ficha em branco, o `__init__` é o preenchimento, e cada parte do processo é uma ficha preenchida.

A classe do raspador

Insper

De que tipo é tjsp

```
import juscraper as jus
tjsp = jus.scraper("tjsp")
print(type(tjsp))
```

Rodem. O que aparece?

tjsp é um TJSPScraper

RESPOSTA

```
<class 'juscraper.courts.tjsp.client.TJSPScraper'>
```

Alguém escreveu a classe `TJSPScraper`, e `jus.scraper("tjsp")` criou um objeto dela, como `Cachorro("Rex", 3)` cria um cachorro.

A classe TJSPScraper, no código do juscraper

Resumo das linhas que abrem a classe e dois dos métodos dela, sem as anotações de tipo e sem o corpo:

```
class TJSPScraper(EsajSearchScraper):
    def __init__(self, verbose=0, download_path=None,
                 sleep_time=0.5, **kwargs):
        ...
    def cjpg(self, pesquisa="", paginas=None, **kwargs):
        ...
```

A mesma forma do `Cachorro`: `class`, `__init__` e métodos que recebem `self`.

range, pela documentação

Ler a documentação do range

Intervalos

O tipo `range` representa uma sequência imutável de números e é comumente usado para percorrer um número determinado de vezes em um laço `for`.

class **range**(*stop*, /)
class **range**(*start*, *stop*, *step=1*, /)

Os argumentos para o construtor de intervalo devem ser inteiros (`int` embutido ou qualquer objeto que implemente o método especial `__index__()`). Se o argumento *step* for omitido, será usado o padrão `1`. Se o argumento *start* for omitido, será usado o padrão `0`. Se *step* for zero, uma exceção `ValueError` será levantada.

```
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

Pela documentação, que páginas `range(1, 11)` pede ao TJSP?

range começa no primeiro número e para antes do segundo

RESPOSTA

```
list(range(1, 11))
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

As páginas de 1 a 10. E a documentação diz `class range`: o `range` se chama como uma função, e é um tipo, como o `Cachorro`. `range(1, 11)` cria um objeto dele.

O programa do juscraper, de novo

```
import juscraper as jus

tjsp = jus.scraper("tjsp")
dados = tjsp.cjpg("golpe do pix", paginas=range(1, 11))

dados.shape
contagem = dados["assunto"].value_counts()
contagem.head(5).plot(kind="barh")
```

Leiam de novo. O que deixou de ser mistério?

O que já dá para ler

Animação no navegador: `animacao-leitura-juscraper.html`, cenário `primeira-volta`.